Week 6 – Lab 4: Execute the Reproduction

Group: G35
Project: P34 – Java Buildability across JDK Versions

Work Completed

Our group is reproducing part of the study *Local Software Buildability across Java Versions*.

We selected 20 Java projects to test across JDK versions 11, 17, 21 and 23, with each group member responsible for four projects.

We downloaded the original researchers' repository and dataset and selected 20 Java projects, with four assigned to each group member. We verified the Git commits to ensure we were using the correct project versions.

We set up Ubuntu and Docker to test the projects using JDK 11, 17, 21 and 23. We also made some necessary changes to the Dockerfile and build scripts to resolve setup issues.

We created a Python script based on the original researchers' code to automatically test each project across the four JDK versions and record the results.

Results

Thelma

Project	**JDK 11**	**JDK 17**	**JDK 21**	**JDK 23**
nextron	**Fail**	**Pass**	**Pass**	**Fail**
equationexploit	**Pass**	**Pass**	**Pass**	**Pass**
xiaofanshu	**Pass**	**Pass**	**Fail**	**Fail**
cmake4eclipse	**Fail**	**Pass**	**Pass**	**Pass**

Out of 16 builds, 11 passed and 5 failed, giving a 68.75% success rate. JDK 17 successfully built all four projects.

All 16 results matched the original researchers' results for the same projects and commits.

Why my builds failed:

nextron (JDK 11): The project is configured to compile Java 17 code, but JDK 11 cannot support that target version. The error was "invalid target release: 17". Switching to JDK 17 fixes the build.

nextron (JDK 23): The build failed during compilation because Java could not find methods such as getInstance() and getPrefix(). This may be caused by Lombok annotation processing not running on JDK 23. The exact cause has not been fully confirmed. The project builds successfully on JDK 17 and 21.

xiaofanshu (JDK 21): The build failed because a compilation tool tried to access an internal Java compiler field that was no longer compatible. The error was NoSuchFieldError: JCTree$JCImport.qualid. This may be related to an older annotation processor.

xiaofanshu (JDK 23): The build failed with 16 compilation errors in the common-base module. Java could not infer type arguments for Result<>. The underlying cause has not been fully confirmed.

cmake4eclipse (JDK 11): An Eclipse Tycho component requires Java 17, but the build was run using JDK 11. The error was UnsupportedClassVersionError. Switching to JDK 17 fixes the build.

Denzel

Project	**JDK 11**	**JDK 17**	**JDK 21**	**JDK 23**
ripme	**Pass**	**Fail**	**Fail**	**Fail**
jar-analyzer-gui	**Fail**	**Fail**	**Fail**	**Fail**
super-xray	**Fail**	**Fail**	**Fail**	**Fail**
jiron-cloud	**Pass**	**Pass**	**Pass**	**Fail**

Out of 16 builds, 4 passed and 12 failed, giving a 25% success rate. JDK 11 built the most projects (2 out of 4) and JDK 23 built none. All 16 results matched the original study's results for the same projects and commits.

Why my builds failed:

ripme (failed on JDK 17, 21 and 23): caused by the Java version. The project is set up to compile as Java 6 code. JDK 12 and newer no longer support compiling for Java 6. The error was "Source option 6 is no longer supported. Use 7 or later" on JDK 17, and "Use 8 or later" on JDK 21 and 23. It builds on JDK 11, so switching JDK fixes it.

jar-analyzer-gui (failed on all four JDKs): not caused by the Java version. Maven could not resolve two of the project's own jar files (lib and lib). The jars are in the project's lib folder, but the pom is set up so Maven checks for them before it installs them. The error is the same on every JDK and happens before any code is compiled, so switching JDK doesn't help.

super-xray (failed on all four JDKs): not caused by the Java version. The project uses the "assembly" goal from the maven-assembly-plugin, which was removed in newer versions of that plugin (3.7.1 was used). The error is the same on every JDK, so switching JDK doesn't help.

jiron-cloud (failed on JDK 23 only): caused by the Java version. The build fails at compilation with "cannot find symbol" errors in the jiron-common-translation module. The same code builds on JDK 11, 17 and 21, so the problem comes from moving to JDK 23. The exact cause is still being checked. Switching to JDK 21 fixes it.

So 2 of my 4 JDK 23 failures (ripme and jiron-cloud) can be fixed just by using an earlier JDK. The other 2 fail because of project setup problems, whatever JDK is used.

Megan

Results:

Project	**JDK 11**	**JDK 17**	**JDK 21**	**JDK 23**
lorne_core	**PASS**	**PASS**	**FAIL**	**FAIL**
loc_unit	**PASS**	**PASS**	**PASS**	**PASS**
nodejs	**FAIL**	**FAIL**	**PASS**	**FAIL**
Hopper_optimization	**PASS**	**FAIL**	**FAIL**	**FAIL**

Out of 16 builds, 8 passed and 8 failed, resulting in a success rate of 50%. JDK 11 successfully built three projects, while JDKs 17 and 21 each successfully built two projects, and JDK 23 successfully built one project. All 16 results matched the original study for the same projects, commits and JDK versions.

Why my builds failed:

lorne_core(failed for JDK 21 and 23) : the project uses outdated Java 7 source and target settings hat the compiler rejects

nodejs(failed for JDK 11, 17 and 23) : The project requires 21 or newer and compiler targets 21 while preview features are enabled under JDK 23. Preview features must target the JDK version being used.

hopperoptimizations (failed for JDK 17,21,and 23) : groovy runtime fails during initialisations while Gradle is starting.

Lucas

Results:

Project	**JDK 11**	**JDK 17**	**JDK 21**	**JDK 23**
pingbypass-client	**Pass**	**Fail**	**Fail**	**Fail**
SignedVelocity	**Pass**	**Pass**	**Pass**	**Pass**
tyme4j	**Pass**	**Fail**	**Fail**	**Fail**
xjavafxtool-spring	**Fail**	**Fail**	**Pass**	**Pass**

Out of 16 builds, 8 passed and 8 failed, giving a 50% success rate. JDK 11 built three of the four projects. All 16 results matched the original researchers' results for the same projects and commits.

Why my builds failed:

smart-medicine (JDK 17 and 21): The build failed during compilation because a compilation tool tried to access internal Java compiler code that newer JDKs no longer allow access to. The error was "module jdk.compiler does not opens com.sun.tools.javac.processing" (ExceptionInInitializerError). The project builds successfully on JDK 11.

smart-medicine (JDK 23): The build failed with compilation errors in HistoryService.java, where Java reported "cannot find symbol" and "invalid method reference".

pingbypass-client (JDK 17, 21 and 23): The project uses Gradle 4.9, which is too old to run on Java 17 or newer. Gradle failed to start before the project was compiled. The error was "Unable to start the daemon process" (ExceptionInInitializerError). The project builds successfully on JDK 11.

SignedVelocity (JDK 11): The Fabric Loom build plugin requires Java 17, but the build was run using JDK 11. The error was "Dependency requires at least JVM runtime version 17. This build uses a Java 11 JVM."

SignedVelocity (JDK 17): The project targets Minecraft 1.21.2, which requires Java 21. The error was “Minecraft 1.21.2 requires Java 21 but Gradle is using 17”. Switching to JDK 21 fixes the build.

Omololuwa

Results:

Project	JDK 11	JDK 17	JDK 21	JDK 23
spring-boot-seed	**PASS**	**PASS**	**PASS**	**FAIL**
xjavafxtool-spring	**FAIL**	**FAIL**	**FAIL**	**FAIL**
ssgwt-lib	**FAIL**	**FAIL**	**FAIL**	**FAIL**
tyme4j	**FAIL**	**FAIL**	**FAIL**	**FAIL**

Out of 16 builds, 3 passed and 13 failed, giving an 18.75% success rate. All 16 results matched the original study's results for the same projects and commits.

Why my builds failed:

spring-boot-seed - Passes on JDK 11, 17 and 21 and fails only on JDK 23. The build stops in the mc-core module, because JDK 23 refuses to read aspectjweaver-1.8.13.jar. The jar has a malformed zip64 header (“Invalid CEN header”), which older JDKs tolerate and JDK 23 rejects. Every “cannot access” and “cannot find symbol” error that follows comes from that one unreadable jar. This is a genuine JDK 23 incompatibility, caused by an old dependency.

xjavafxtool-spring - Fails on all four JDKs before anything is compiled. Maven refuses to load the project’s pom.xml because the repository id xwintop-maven is declared twice (line 36), pointing at two different URLs. This is a defect in the project’s own POM, so no JDK version can fix it. I saw this error in the JDK 11 log, and the authors’ dataset shows the project failing on every Java version from 6 to 23, which is consistent with it.

ssgwt-lib - Fails on all four JDKs because the study’s automation can’t build it. The runner tries the Ant targets clean, jar, war and dist in turn. clean succeeds, but the other three don’t exist in the project’s build.xml, so nothing is ever compiled. It is a limitation of how the study’s script handles this project, not evidence that the code is incompatible with any JDK. I saw this in the JDK 11 log, and the dataset also shows a failure on every Java version.

Tyme4j - Fails on all four JDKs even though the code is fine. Compilation succeeds and all 395 tests pass, and the jar is built. The failure comes afterwards, in maven-javadoc-plugin:2.9, which looks for javadoc at /opt/java/../bin/javadoc. That path only exists in the JDK 8 directory layout, so the old plugin can't find the tool on JDK 9 or later. I confirmed the same error on all four JDKs. The dataset agrees, since tyme4j passes on Java 8 and fails from Java 9 up.